Лекция 05 — Семантика перемещения

Содержание

1. Категории значений
2. Проблема избыточного копирования
3. Семантика перемещения
    1. Rvalue-ссылки
    2. Перемещающие методы
    3. `std::move`
4. Универсальные ссылки
5. Copy elision
6. Принципы и идиомы

”Всё тлен. Всё бесполезно.”
— А. П. Хвастунов

Категории значений

Каждое выражение в C++ характеризуется двумя независимыми свойствами: типом и *категорией значения* (*value category*), которая определяет, как компилятор может распоряжаться данными.

В незапямятные времена языка Си говорили лишь о двух категориях: lvalue и rvalue. Выражения типа lvalue всегда обозначали объект, то есть именованную область памяти; rvalue определялись методом исключения как всё, что не является lvalue. Названы они так из-за того, что по левую сторону знака равенства всегда стоял операнд типа lvalue, а операнд типа rvalue всегда стоял лишь по правую его сторону; впрочем, под lvalue могут понимать не только *left-hand side value*, но и *locator value*.

В C++ эта структура ощутимо усложнилась и на сегодняшний день формирует полноценную таксономию выражений (*ровно так это называет Стандарт*). Всего выделяется пять основных категорий значений:

> I. `glvalue`,
> или *«generalized» lvalue*.
>
> Это композитная категория, включающая все выражения, вычисление которых определяет идентичность объекта или функции; иными словами, имеющие

идентифицируемый адрес в памяти.

II. `prvalue`,
или *pure rvalue*.

Это выражение, вычисление которого инициализирует временный объект. Строго говоря, prvalue является всё из следующего списка:

- литералы (за исключением строкового);
- вызов функции, не возвращающей ссылку;
- выражения, состоящие из наброа операторов и задающие какой-либо новый объект;
- преобразующие выражения;
- указатель `this`;
- `enum`;
- шаблонные параметры скалярного типа;
- лямбды;
- выражения `requires` и специализации концептов.

III. `xvalue`,
или *eXpriring value*.

Это glvalue, ресурс которого может быть переиспользован. Строго говоря, xvalue является всё из следующего списка:

- член rvalue-объекта или указатель на него;
- вызов функции, оператора или преобразования, если результирующее значение — это rvalue-ссылка (*см. Семантика перемещения*);
- любое выражение, приводящее ко временной материализации prvalue. [1]

Выражения типа xvalue одновременно обладают и свойствами glvalue, и свойствами rvalue; практический смысл этого утверждения станет ясен далее.

IV. `lvalue`,
или *left-hand side value*.

Это glvalue, не являющиеся xvalue; иными словами, их данные нельзя переиспользовать.
Строго говоря, lvalue является всё из следующего списка:

- имена переменных, функций, шаблонных параметров или членов класса;
- вызов функции, возвращающей ссылку;

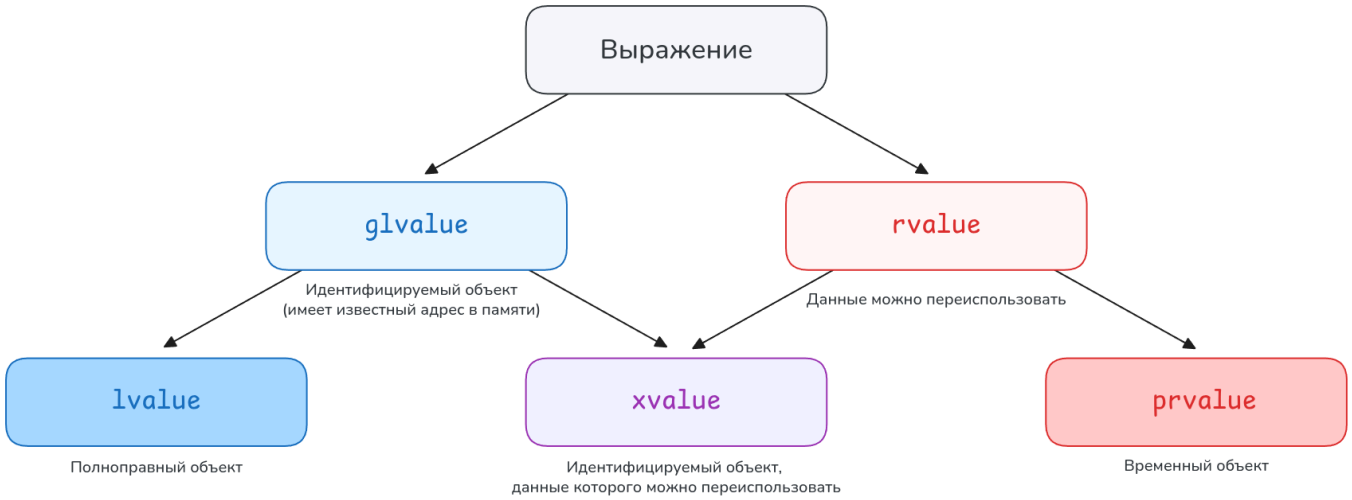
- выражения, состоящие из набора операторов и изменяющие существующий объект;
- строковые литералы;
- преобразования к ссылке;
- шаблонный параметр ссылочного типа.

Одно из ключевых свойств lvalue — у них можно взять адрес при помощи оператора амперсанд. Как было отмечено ранее, они также могут находиться слева от оператора присваивания, а также ими можно инициализировать lvalue-ссылки; словом, lvalue — это обычный полноправный объект.

V. `rvalue`,
или *right-hand side value*.

Это композитная категория, объединяющая prvalue и xvalue. Свойства таких выражений диаметральны свойствам lvalue: мы не можем взять их адрес, поставить их слева от оператора присваивания или инициализировать ими неконстантную lvalue-ссылку.

Для простоты понимания эту структуру можно визуализировать следующим образом:



Важное замечание состоит в том, что некоторые из предыдущих формулировок могут показаться слишком абстрактными, что совершенно справедливо; в частности, нам следует пояснить, что мы подразумеваем под «переиспользованием» и зачем нам вообще понадобилась такая сложная таксономия.

Проблема избыточного копирования

Рассмотрим простой класс, владеющий некоторыми данными — например, обертку над статическим массивом. Нас в этом случае беспокоит единственный метод — конструктор копирования.

Как бы правильно он ни был написан, с ним все еще связана одна загвоздка. Допустим, у нас также имеется некоторая функция-фабрика `CreateArray`, с помощью которой мы хотим создать экземпляр массива:

```
int main() {
    CArray arr = CreateArray();
}
```

Проанализируем этот код с точки зрения жизненного цикла объектов. Сначала `CreateArray` создаёт временный объект, он копируется в `arr`, после чего удаляется, и выполнение программы продолжается. Проблема здесь в первую очередь в том, что мы совершаем копирование, которого можно избежать. И мы даже совершенно точно знаем, как этого лучше всего избежать, ибо ровно эту идею уже эксплуатировали в идиоме copy-and-swap: мы можем просто поменять два объекта местами, поскольку один из них нам гарантированно не нужен. В этот момент нам становятся ясны две вещи.

Во-первых, теперь понятно, что подразумевалось под переиспользованием данных: мы можем «забрать» их у объекта, которому они больше не нужны. Более того, теперь понятно и разделение на `prvalue` и `xvalue`, поскольку переиспользовать данные временных объектов можно по умолчанию, а `xvalue` является результатом того, что мы наделили этим свойством какой-то существовавший до этого объет, хотя пока и неясно, как именно мы это делаем.

Во-вторых, теперь понятно, зачем нам вообще нужна такая неочевидная организация категорий выражений. Если мы сможем каким-то образом реализовать отдельный механизм для копирования `rvalue` выражений, то получим огромный прирост в производительности при работе со временными объектами, а также эффективный инструмент для передачи владения существующими данными.

Семантика перемещения

Rvalue-ссылки

Скорее всего, для читателя не является секретом тот факт, что помимо lvalue-ссылок в C++ существует и такое понятие, как rvalue-ссылка.

Rvalue-ссылки появились вместе с C++11. Они, как и константные lvalue-ссылки, могут быть использованы для того, чтобы продлять жизнь временным переменным, что, как показывает практика, необходимо в первую очередь при передаче их в функцию. Отличительная черта rvalue-ссылок в том, что они могут определяться *только* rvalue-объектами и, что наиболее важно, в процессе разрешения перегрузок они выбираются как наиболее точное соответствие rvalue-аргументу:

```
void f(int&& x) {
    std::cout << "Rvalue ref\n";
}

void f(const int& x) {
    std::cout << "Const lvalue ref\n";
}

int main() {
    f(3104); // Rvalue ref!
}
```

Это делает их нашим главным инструментом в рамках текущего обсуждения: они позволяют создать отдельный независимый канал для объектов, владение ресурсом которых можно передать.

Перемещающие методы

Наличие механизма rvalue-ссылок очень удачно вписывается в контекст поставленной проблемы, ибо теперь мы можем явно указать конструктор «копирования», который будет вызываться *только* для временных объектов:

```
class CArray {
    // ...
    CArray(CArray&& other)
        : size_(std::exchange(other.size_, 0))
        , data_(std::exchange(other.data_, nullptr))
    {}
    // ...
}
```

Теперь в случае, когда мы захотим проинициализировать новый объект временным значением, нам не придется делать никаких тяжелых копирований: мы можем просто перенести все данные как есть.

Слово «копирование» выше взято в кавычки неспроста — то, что мы определили, называется **конструктором перемещения** (*move constructor*), и его название совершенно точно отражает его семантику: мы буквально взяли данные и переместили их из одного места в другое.

> ⚠ **Важно!**
>
> После перемещения объект должен остаться в *валидном, но неопределённом состоянии*, то есть прежде всего сохранить свой инвариант.

Это критически важно осознать. Такое требование связано с тем, что перемещённый объект всё ещё необходимо уничтожить, а также с тем, что объекты категории xvalue после перемещения должны допускать задание нового значения. Более того, не рекомендуется делать с перемещёнными объектами что-либо, кроме этих двух операций.

Отдельно стоит сказать и про `std::exchange`, который является самым чистым и наиболее идиоматическим способом реализации перемещающих методов. Он возвращает старое значение переменной и заменяет его новым, практически «вымещая» его, что делает логику перемещения максимально прозрачной. В отличие от `std::swap`, он также позволяет использовать список инициализации полей и иногда работает чуть шустрее. Кстати говоря, как `std::exchange`, так и `std::swap` ожидаемо реализованы при помощи перемещения объектов, а не копирования.

Поскольку мы объявили перемещающий конструктор, логично будет объявить и парный ему *оператор перемещающего присваивания*:

```
class CArray {
    // ...
    CArray& operator=(CArray&& other) {
        delete[] data_;
        size_ = std::exchange(other.size_, 0);
        data_ = std::exchange(other.data_, nullptr);
        return *this;
    }
    // ...
}
```

Здесь также стоит сразу напомнить о том, что с приходом этих двух методов хорошо известное нам правило трёх расширяется до **правила пяти**:

> ⚠ **Правило пяти**

Если в классе был объявлен хотя бы один из пяти следующих специальных методов, то необходимо объявить и оставшиеся:

1. Деструктор
2. Конструктор копирования
3. Оператор копирующего присваивания
4. Конструктор перемещения
5. Оператор перемещающего присваивания

Иногда также встречается и обратная формулировка: если удалён хотя бы один из методов, необходимо удалить все.

Не следует забывать и о *правиле нуля*, которое предписывает нам избегать переопределения этих методов для классов, не владеющих данными.

Другой момент заключается в том, что при перемещении исходный объект *всегда* меняет своё состояние. Менее важное замечание: нет никакого смысла принимать готовые к перемещению объекты по константной rvalue-ссылке или пытаться переместить константные объекты. Более важное замечание:

⚠ **Важно!**

Перемещающий конструктор и оператор перемещающего присваивания **обязаны** быть помечены `noexcept`.

Это критично для контейнеров, поддерживающих строгую гарантию исключений, то есть, в частности, практически всех стандартных. Очень показательным и поучительным примером будет следующий код:

```
struct HeavyObject {
    HeavyObject() = default;
    HeavyObject(HeavyObject&& other) {
        std::cout << "Move constructor\n";
    }

    HeavyObject(const HeavyObject& other) {
        std::cout << "Copy constructor\n";
    }
};

int main() {
```

```
    std::vector<HeavyObject> vec;
    vec.push_back(HeavyObject());
    vec.push_back(HeavyObject());
}
```

Анекдот в том, что для перемещения старых данных после реаллокации памяти в векторе будет использован конструктор копирования, а не перемещения, ибо если конструктор перемещения выбросит исключение, то строгая гарантия нарушится. Вектор в этом случае ставит безопасность выше, чем производительность, и отдаёт приоритет методу, который гарантирует сохранность данных. Таким образом, отсутствие незначительного на первый взгляд квалификатора может полностью угробить производительность программы и лишить смысла всё, ради чего эта авантюра с перемещением вообще затевалась.

Кстати, что там в самом деле с производительностью?
Согласно некоторым бенчмаркам, правильно реализованное перемещение происходит примерно всемеро быстрее, чем копирование при той же рабочей нагрузке. При неверно реализованной семантике перемещения миллисекундная операция может растянуться на 7 миллисекунд. В крупномасштабном сценарии разница такова: можно обработать 1 000 запросов в секунду, а можно 140 запросов в секунду.

Подытожить этот блок следует не авторскими, но очень важными словами: семантика перемещения — это не просто оптимизация, а фундаментально новый подход к трактовке времени жизни объекта и к владению данными. Достаточно глубокое её понимание позволяет не только писать эффективный код на C++, но и грамотнее рассуждать об управлении ресурсами на любом языке программирования в целом.

`std::move`

Перемещающие методы служат главным инструментом для эффективной передачи владения ресурсом от одного объекта к другому; тем не менее, мы до сих пор не знаем, как указать компилятору, что время этого владения вышло или, иначе говоря, что некогда lvalue-объект теперь является xvalue.

Для этого в язык была введена шаблонная функция `std::move`, которая практически представляет собой ровно выражение вида `static_cast<T&&>(t)`, то есть явное приведение к rvalue-ссылке. Вот как с её помощью решается наша задача:

```
struct CArray {
    CArray() {
        std::cout << "Constructor\n";
    }
```

```
    CArray(const CArray& other) {
        std::cout << "Copy constructor\n";
    }

    CArray(CArray&& other) {
        std::cout << "Move constructor\n";
    }

    CArray& operator=(const CArray& other) {
        std::cout << "Copy assignment operator\n";
        return *this;
    }

    CArray& operator=(CArray&& other) {
        std::cout << "Move assignment operator\n";
        return *this;
    }
};

int main() {
     CArray arr1;
    CArray arr2;
    arr1 = std::move(arr2);
}

// Out;
// Constructor
// Constructor
// Move assignment operator

// Если бы мы не использовали std::move, вызвался бы copy assignment
operator
```

> ⚠ **Важно!**
>
> Очень важно держать в уме, что `std::move` **ничего не перемещает** — в этом плане его название, вообще говоря, выбрано не очень удачно. Он лишь приводит объект в состояние *готовности к перемещению*, но непосредственно передача владения произойдёт далее, когда результатом вызова `std::move` воспользуется кто-то другой.

Существует также её версия `std::move_if_noexcept`, на которую полагаются стандартные контейнеры при перемещении внутренних данных.

Универсальные ссылки

Мы решили проблему с тем, чтобы разделить обработку `lvalue` и `rvalue`; тем не менее, конкретным способом этого разделения на текущем этапе выступает обычная перегрузка, и если это не совсем критично в случае одного аргумента, то в случае всего трёх аргументов перегрузок уже понадобится 8; словом, их количество растет экспоненциально при росте количества аргументов. И хотя нашей исходной мотивацией действительно было иметь явный отдельный канал для работы с rvalue объектами, такое досадное обстоятельство заставляет нас задуматься о том, чтобы завести единый «умный» синтаксис передачи значений в функцию, который сохранял бы их категорию.

Прежде всего следует обозначить такое свойство ссылок, как *схлопывание* (*reference collapsing*): при определенных условиях мы можем объявлять «ссылки на ссылки», в результате чего во всех случаях, кроме rvalue-ссылки на rvalue-ссылку, будет получаться lvalue-ссылка:

```
using lref = int&;
using rref = int&&;

int n = 3104;
lref& r1 = n; // int&
lref&& r2 = n; // int&
rref& r3 = n; // int&
rref&& r4 = n; // int&&
```

Если теперь мы передадим в шаблонную функцию, принимающую rvalue-ссылку `T&&`, в качестве шаблонного параметра `T` lvalue-ссылку, то ссылочные квалификаторы схлопнутся, превратив параметр функции в обычную lvalue-ссылку; иначе же он останется rvalue-ссылкой. Единственная проблема такого подхода в том, что мы обязаны приводить объект к ссылке вместо того, чтобы передать его по значению, как мы делаем это при работе с обычными функциями.

Именно в этот момент появляется понятие **универсальной ссылки**. Это не столько отдельная сущность, сколько особое *правило вывода типа*: если шаблонная функция принимает в качестве аргумента rvalue-ссылку от шаблонного параметра, то при передаче в неё объекта категории lvalue его тип автоматически преобразуется в соответствующую lvalue-ссылку; иными словами, универсальная ссылка сохраняет категорию объекта, на который указывает:

```
template<typename T>
void Test(T&& val) {
```

```
    std::cout << std::is_same_v<T, int&> << std::endl;
    std::cout << std::is_same_v<T, int> << std::endl;
}

int main() {
    int x = 238;
    Test(3104); // 0 1
    Test(x);    // 1 0
}
```

Стоит также отметить, что универсальные ссылки допустимо объявлять при помощи синтаксиса `auto&&`.

Хотя мы и получили единый канал связи с функцией, наши методы перемещения и копирования всё ещё рассчитаны на раздельную обработку: мы не можем просто слепо использовать `std::move` для объектов, переданных по универсальной ссылке — нам требуется обращаться к разным механизмам в зависимости от того, какой тип за ней на самом деле хранится, и в этом нам помогает `std::forward`, который является последним шагом на нашем непростом пути.

`std::forward` реализует «условное» перемещение в зависимости от типа шаблонного параметра, сохраняя его категорию; говоря иначе, он позволяет переместить rvalue и скопировать lvalue. Мы никогда не можем определить исходную категорию объекта по типу универсальной ссылки, ибо она всегда имеет тип `T&&`; статус-кво объекта зафиксирован лишь в шаблонном параметре `T`. Это, в частности, означает, что при работе с `std::forward` мы обязаны этот самый параметр указывать, тогда как при работе с `std::move` мы, наоборот, скорее не должны этого делать.

Итак, решение, к которому мы приходим с `std::forward`, выглядит следующим образом:

```
void Process(int&& value) {
    std::cout << "Rvalue overload\n";
}

void Process(const int& value) {
    std::cout << "Lvalue overload\n";
}

template<typename T>
void Dispatch(T&& value) {
    Process(std::forward<T>(value));
}

int main() {
```

```
    int x = 238;
    Dispatch(x);    // Lvalue overload
    Dispatch(3104); // Rvalue overload
}
```

Совокупность описанных выше механизмов называется *идеальной передачей* (*perfect forwarding*), и с точки зрения нашей исходной задачи эта механика действительно лишена изъянов.

> ⚠ **Важно!**
>
> Прибегать к использованию `std::forward` следует только в рамках идеальной передачи, ибо он был придуман специально для неё; в обычном коде, когда нам необходимо переместить откуда-то куда-то какое-то значение, необходимо использовать `std::move`.

В C++23 также была введена функция `std::forward_like`, которая «проецирует» категорию одного объекта на другой. Она бывает уместна, например, при работе с универсальными обёртками в паре с deducing this; более подробно разобрана она не будет, ибо это находится не то слишком продвинутым, не то слишком бесполезным.

Copy elision

Если мы вернемся к самому первому примеру с `CreateArray` и решим на практике выяснить, какую последовательность вызовов совершит компилятором, то, скорее всего, с удивлением обнаружим, что она не соответствует той, которую мы ожидаем:

```
struct CArray {
    CArray() {
        std::cout << "Constructor\n";
    }

    CArray(const CArray& other) {
        std::cout << "Copy constructor";
    }

    ~CArray() {
        std::cout << "Destructor\n";
    }
};
```

```
CArray CreateArray() {
    return CArray();
}

int main() {
    CArray arr = CreateArray();
}
```

Этот код ровно единожды вызовет конструктор и деструктор, без каких-либо копирований или инициализации временных объектов. Почему так происходит?

Дело в том, что при определенных обстоятельстах компилятор имеет право опускать копирование или перемещение временных объектов и конструировать их сразу в том месте, куда они были бы в итоге скопированы или перемещены соответственно. Это допустимо в следующих ситуциях и формах:

1. если объект инициализируется временным значением того же типа; причем если это происходит в результате вызова функции (как в примере выше), то такая оптимизация называется **RVO** (*Return Value Optimization*);
2. если функция возвращает именованную локальную переменную, тип которой совпадает с возвращаемым типом функции; такая оптимизация называется **NRVO** (*Named Return Value Optimization*).

Начиная с C++17 RVO стала обязательной. NRVO сложнее и формально остаётся на усмотрение компилятора, хотя почти всегда поддерживается. Даже если NRVO не применяется, компилятор всё ещё может автоматически переместить объект.

Следует понимать две основные вещи, связанные с этими оптимизацями.

Во-первых, это единственный во всём языке случай, когда компилятору дозволено игнорировать любые глобальные побочные эффекты. Из этого можно вынести один главный вывод — не нужно писать конструкторы так, чтобы они имели глобальные побочные эффекты.

Во-вторых, ни в коем случае не следует использовать конструкцию вида `return std::move...`, ибо это препятствует как RVO, так и NRVO. Может показаться, что мы меняем копирование на перемещение и тем самым выигрываем в производительности, но на самом деле мы меняем ничего на перемещение:

```
struct CArray {
    CArray() {
        std::cout << "Constructor\n";
    }
```

```
    CArray(CArray&& other) {
        std::cout << "Move constructor\n";
    }

    ~CArray() {
        std::cout << "Destructor\n";
    }
};

CArray CreateArray() {
    return std::move(CArray());
    // так НЕ НАДО
}

int main() {
    CArray arr = CreateArray();
}

// Output:
// Constructor
// Move constructor
// Destructor
// Destructor
```

Источники

1. А. П. Хвастунов — Лекции по основам программирования на C++, 2 семестр, 2026
2. cppreference.com — Value categories (https://en.cppreference.com/cpp/language/value_category)
3. cppreference.com — Reference declaration (https://en.cppreference.com/w/cpp/language/reference.html)
4. cppreference.com — Copy elision (https://en.cppreference.com/cpp/language/copy_elision)
5. @MGNeo — Copy elision, или как отстрелить ногу по самую шею... (https://habr.com/ru/articles/277333/_)
   Примечание. Не очень полезно, но любопытно как дискуссия.
6. Ahmed Farid — std::move ничего никуда не двигает: подробный рассказ о категориях значений в C++ (Перевод) (https://habr.com/ru/articles/985066/)
   Примечание. Великолепная статья! Из неё было позаимствовано множество практических принципов. Крайне рекомендуется к прочтению, если хочется понять семантику перемещения максимально глубоко и разобрать её на реальных примерах.

1. см. раздел Temporary materialization: https://tcs.rwth-aachen.de/docs/cpp/reference/en.cppreference.com/w/cpp/language/implicit_cast.html ↩